# DyTrustRec System Architecture

*A Detailed Walkthrough of the Full Pipeline Diagram*

This document takes the single full-system architecture diagram that summarises DyTrustRec and walks through it in enough depth that a reader can follow every box, every arrow and every design decision without reading the source code. It is a companion to the research paper, not a replacement for it: the paper argues the claims and reports the results; this document explains the machine that produced them.

The diagram appears in full on the next page, sized as large as a single page allows. The sections after it follow the diagram region by region, in the order the data moves: top to bottom, left to right.

## How to Read the Diagram

Every box carries a module number where one applies. The numbering follows the project's internal module inventory, which is why it is not consecutive: modules 9, 10, 19, 21 and others exist in the codebase but sit outside the part of the pipeline this figure covers. Dashed orange rectangles group boxes into named sub-stages. A solid arrow is a direct, forward data dependency: the box at the arrowhead consumes the output of the box at the tail. A dashed arrow marks a relationship that exists during training only. The diagram has two of them: the dashed line from the encoder's output into the training-objective box, and the dashed line along the top of the first stage that marks the EDA branch as informational.

Colour is used consistently rather than decoratively:

| | |
|---|---|
| **Orange** | A core processing module: something that transforms data on the main path (ingestion, cleaning, the two review encoders, the behavioural trust detector, the anomaly-to-trust conversion). |
| **Grey** | A structural or configuration object rather than a learned component: the chronological split, the raw text fields, the five hand-built structured features. |
| **Red** | An enforcement stage. Only Leakage Guards is red: it exists to catch a mistake, not to produce data. |
| **Gold / tan** | An analysis or output-facing module: EDA and the heterogeneous graph itself. |
| **Blue** | A learned neural layer with trainable weights: the fusion MLP and the HGT's type-specific projection. |
| **Green** | A load-bearing output artefact: the sample weights, the learned embeddings and the final top-K recommendation. |

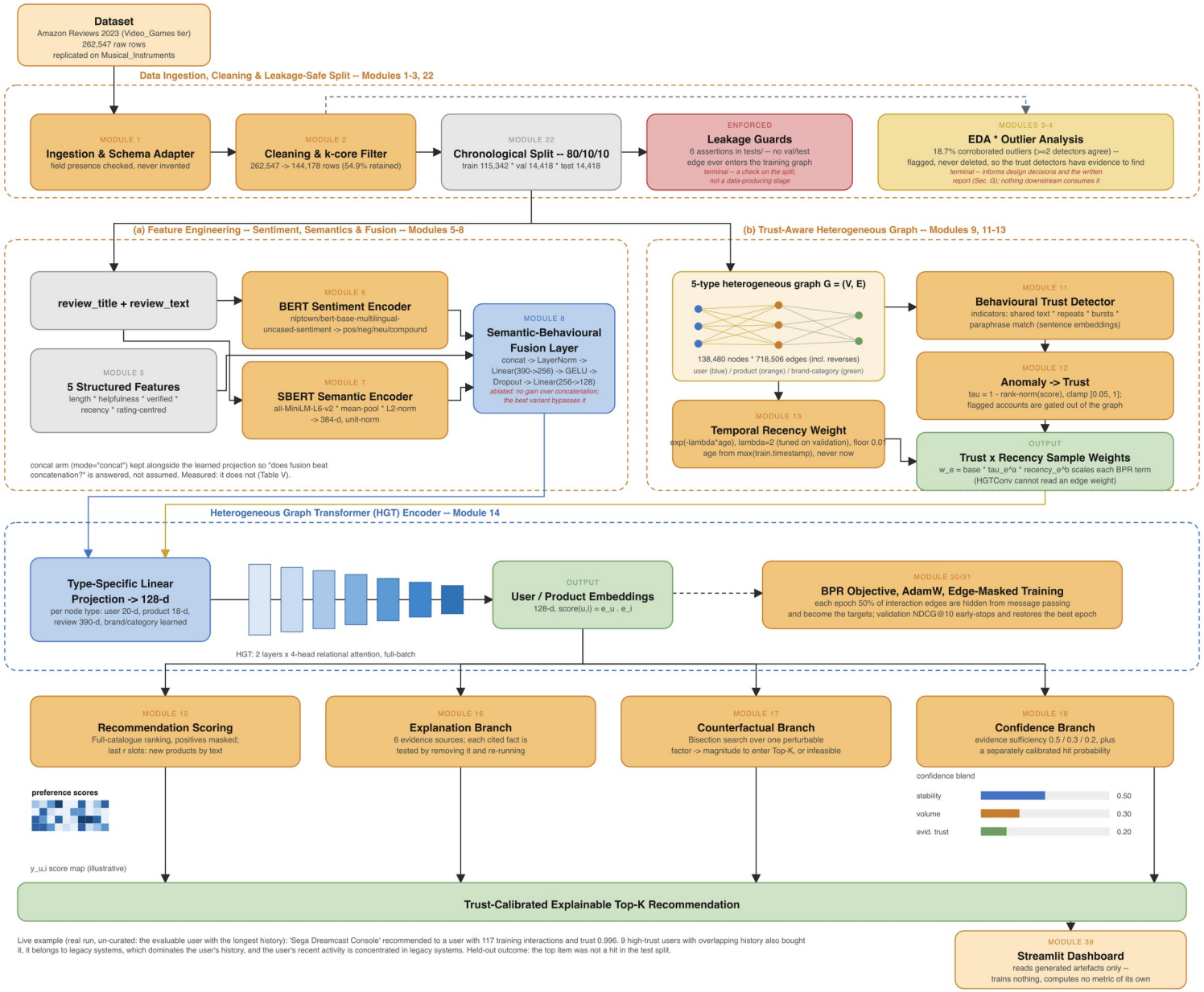



*Fig. 1. The complete DyTrustRec pipeline, from raw review ingestion through the trust-calibrated, explainable top-K output.*

# 1. Stage 0: The Dataset

**Dataset** is the box at the top left and the only input to the whole system. It is drawn in the same peach tone as the Streamlit Dashboard at the bottom right, marking both as endpoints rather than processing stages: one is where real data enters, the other is where a finished result is consumed. The diagram records the exact slice used: the Video_Games tier of Amazon Reviews 2023, 262,547 raw rows before any cleaning. Every number elsewhere in the diagram traces back to a transformation of these rows. The same pipeline was later run unchanged on a second category, Musical_Instruments (281,003 raw rows), as a replication; the architecture is identical, only the data and the tuned settings differ, so the diagram shows the first dataset's counts and notes the second in the box.

Data collection works from CSV. The raw JSON review and metadata files are converted once to flat CSV (list and dictionary cells stored as JSON text and decoded on read), and every later stage reads the CSV.

# 2. Stage 1: Data Ingestion, Cleaning and the Leakage-Safe Split

The widest dashed container holds five boxes in a row: two that transform the data in sequence, one that partitions it, and two (Leakage Guards and EDA) that consume the result without producing anything that feeds forward. The arrow that continues the pipeline downward comes out of the Chronological Split box, not out of Leakage Guards or EDA; those two are siblings of the split, not a continuation of the chain.

## 2.1 Module 1: Ingestion and Schema Adapter

The first box maps whatever field names the source uses onto a fixed, canonical schema. Its caption, "field presence checked, never invented", describes a deliberate behaviour: if a field is declared in the schema but absent from the source, the adapter records it as absent rather than filling in a placeholder. Amazon Reviews 2023 has no seller field, which is why the graph built later has five node types rather than the six first planned.

## 2.2 Module 2: Cleaning and k-core Filter

This box takes 262,547 rows down to 144,178 (54.9% retained). Two things happen inside it. Records that cannot be interpreted at all (an out-of-range rating, an impossible timestamp, a missing key) are removed. A five-core filter then keeps only users and items with at least five interactions, which accounts for most of the drop. Records that merely look suspicious (near-duplicate text, unusual length, review bursts inside a sixty-second window) are flagged with a reason string and kept. That matters two stages later: if the suspicious rows were deleted here, the trust detectors would have nothing left to find.

## 2.3 Module 22: Chronological Split, 80/10/10

The cleaned rows are partitioned strictly by time: train 115,342, validation 14,418, test 14,418. No training example comes from after a validation or test example in real time. Every downstream arrow originates from this box. A line runs down from it past Leakage Guards and EDA on its way to the two branches below, but the dependency is direct: Split feeds Feature Engineering and Graph Construction, and neither of its right-hand neighbours sits on that path.

## 2.4 Leakage Guards: why it is a dead end

Leakage Guards is red because it is the one enforcement box, and it is annotated "terminal: a check on the split, not a data-producing stage". Concretely it is six assertions in the test suite: the training graph contains

only training-period edges; every leakage-sensitive statistic (scalers, item means, the trust detector itself) is fitted on the training partition and nothing else; the recency reference point is always the maximum training timestamp, never wall-clock time; and negative sampling excludes positives from every split, not just training. Running them produces a pass or a failure, not a dataset, so nothing downstream has anything of the right type to consume.

### 2.5 EDA and Outlier Analysis: why it is also a dead end

EDA is annotated "terminal" for a different reason. It runs several outlier heuristics over the cleaned data and reports that 18.7% of rows are corroborated outliers (at least two independent detectors agree). That statistic is descriptive: it informs the written report and design decisions and is never read back into the pipeline. The caption "flagged, never deleted, so the trust detectors have evidence to find" does not mean the trust detector reads this box. It means that because cleaning flagged suspicious rows instead of deleting them, those rows are still present in the training data, which is what gives the trust detector something to find.

## 3. Stage 2(a): Feature Engineering: Sentiment, Semantics and Fusion

The left container, labelled (a), takes each review's raw text and behavioural metadata and turns them into one fixed-length vector per review, the representation the rest of the pipeline operates on.

### 3.1 Inputs: review text and the five structured features

Two grey boxes feed this container. **review_title + review_text** is the raw text fields concatenated, with no processing yet. **Module 5, five Structured Features**, are cheap hand-computed signals that need no neural encoder: log review length, log helpful-vote count, a verified-purchase flag, recency, and a leakage-safe rating centred on the item's training-only mean. Grey marks these as structural inputs; nothing here has trainable weights.

### 3.2 Module 6: BERT Sentiment Encoder

The text is read once for sentiment by nlptown/bert-base-multilingual-uncased-sentiment, a BERT model fine-tuned on product-review text to predict a one-to-five star rating, remapped onto a pos/neg/neu/compound schema. It replaced an earlier VADER lexicon scorer, because a fine-tuned classifier can pick up negation, sarcasm and mixed sentiment that a fixed word list cannot, at the cost of a forward pass instead of a dictionary lookup. It runs frozen, so it adds no training-data leakage.

### 3.3 Module 7: SBERT Semantic Encoder

The same text is read a second time, independently, for meaning rather than sentiment, using all-MiniLM-L6-v2, a compact Sentence-BERT variant. The output is mean-pooled and L2-normalised into a 384-dimension unit-norm vector. Two encoders trained for two objectives is deliberate: sentiment and topical meaning are different signals.

### 3.4 Module 8: Semantic-Behavioural Fusion Layer

Module 8 is the one fusion point in the diagram. It concatenates the SBERT vector, the sentiment compound score and the structured features into a 390-dimension vector, then passes it through a small learned network (LayerNorm, a 390 to 256 projection, GELU, dropout, a 256 to 128 projection and a final LayerNorm) to produce the 128-dimension representation used per review. The note under the box records a methodological choice: the pipeline can bypass the learned layer and use the raw 390-dimension

concatenation, so that "learned fusion is better" is something the ablation tests and not something the architecture assumes. Measured, it does not: the learned-fusion arm scored below the concatenation arm in both evaluation protocols, so the best variant bypasses this layer, and the box is annotated that way on the diagram.

# 4. Stage 2(b): The Trust-Aware Heterogeneous Graph

The right container, labelled (b), runs in parallel with feature engineering; both read the same split independently. Where (a) turns each review into a vector, (b) turns the whole training table into a graph and works out how far each interaction should be trusted and how recent it is.

## 4.1 The five-type heterogeneous graph

The node-and-edge picture inside the box is a stylised illustration of the structure: blue nodes are users, orange products, green the brand and category types. The real graph has five node types (user, product, review, category, brand) joined by five relation types, each added in both directions so information can flow back toward users, for 138,480 nodes and 718,506 edges including reverses. The box is cream rather than orange because it is a structural object, not a learned transformation.

## 4.2 Module 11: Behavioural Trust Detector

This module replaced CoLA in the production path. CoLA scores a node by how well its attributes can be predicted from its neighbourhood, which is the wrong question for review manipulation: a coordinated campaign is internally consistent by construction, so each shill is easy to predict from the others and is rated as more trustworthy than a genuine user (AUC 0.295 on the synthetic benchmark, below the 0.5 chance line). The behavioural detector asks the question the shilling-detection literature asks: does the account behave like a campaign account? It combines three label-free indicators, each rank-normalised and averaged with equal weight: the share of the user's reviews whose text other accounts also posted, the share that repeat the user's own text, and the most reviews posted inside any 24-hour window. No threshold or weight is tuned. A fourth indicator, distance from the item's mean rating, was tried and dropped because on the benchmark it pointed the wrong way (AUC 0.19 to 0.34).

## 4.3 Module 12: Anomaly to Trust

The detector's output is an unbounded anomaly score, not a probability and not a trust value. This box makes the conversion explicit and configurable: the score is rank-normalised and mapped to $\tau = 1 - \text{rank-norm}(\text{score})$, clamped to [0.05, 1.0]. The floor exists for a concrete reason: a weight of exactly zero would silently delete an interaction from training and change the effective dataset size, which would confound the ablation the weight is meant to inform. The CoLA score is still computed and kept as a documented negative control; the box says so on the diagram.

## 4.4 Module 13: Temporal Recency Weight

Computed independently of trust is a weight based purely on age: $\text{recency} = \exp(-\lambda \cdot \text{age})$, floored at 0.01, with age measured in years from the maximum training timestamp. The decay rate $\lambda$ is selected on the validation split (selected value 2). The reference is never wall-clock time, which would let the weighting drift with the day an experiment happened to run. This is exponential recency weighting, not a temporal graph network, and it is described that way everywhere.

### 4.5 Output: trust and recency sample weights

The green box is where the two weights combine multiplicatively into one per-interaction weight, $w = \text{base} \cdot \tau^{\alpha \cdot \text{recency}}\beta$, with both factors fixed at 1.0 when their ablation flag is off. The weights scale each interaction's term in the training objective. The first version of the project stored them as edge weights on the graph, but the HGT layer cannot consume an edge weight, so those arms were identical to the unweighted model and their score differences were seed noise. That is why the box now says what it does: it weights the loss.

## 5. Stage 3: The Heterogeneous Graph Transformer Encoder

Both branches of stage 2 converge here: the blue line from the Fusion Layer carries the review representations, and the gold line from the weights box carries the sample weights used by the objective. The container has a blue dashed border to mark it as one self-contained module (Module 14).

### 5.1 Type-specific linear projection

Node types arrive with different dimensionality: user profiles are 20-d, product profiles 18-d, review representations 128-d (or 390-d under the concatenation ablation), and category and brand nodes carry only a learned embedding. A type-specific linear layer maps all of them into one 128-dimension space so a single attention mechanism can reason over all five types.

### 5.2 The HGT stack

The shrinking blue blocks are a schematic stand-in for the encoder, not a literal layer count. The real encoder has two relation-aware attention layers with four heads and 128 hidden channels, dropout 0.1. Relation-aware means attention is computed per relation type, so a user–review edge and a product–category edge do not compete through the same parameters. Training is full-graph: one optimiser step per epoch over the whole graph, learning rate 0.01.

### 5.3 Output: user and product embeddings

The green box holds the 128-dimension embedding of every user and product, and the scoring rule is written on the diagram: $\text{score}(u, i) = e_u \cdot e_i$, a plain dot product. Everything below reads only from this box.

### 5.4 Modules 20 and 31: the training objective (edge-masked BPR)

The dashed arrow is the only forward-looking dependency that exists at training time and not at inference. Bayesian Personalised Ranking treats recommendation as pairwise preference learning: an observed interaction should score above an unobserved one. Two details of how it is applied matter more than the loss itself.

- **Edge masking.** The interaction being scored is also a message-passing edge, so an unmasked HGT can read the answer off the graph. The measured symptom was a training loss falling from 0.30 to 0.04 while held-out NDCG stalled and the recommendations spread over 62% of the catalogue: memorisation, not taste. Each epoch a random 50% of the interaction rows are hidden from message passing (all edge types that encode the same row, forward and reverse, share one mask) and those hidden rows are the targets. Evaluation uses the full graph.
- **Validation early stopping.** Held-out NDCG@10 is measured every five epochs; the best weights are restored, and training stops when it has not improved for eight measurements.
- **Weighted loss.** The trust and recency weights scale each BPR term and are rescaled to mean 1, so weighting changes where the gradient goes and not how large it is.

## 6. Stage 4: The Four Output Branches

The trained embeddings feed four parallel branches, each answering a different question about the same recommendation. They read the frozen embeddings and apply fixed logic on top; none has trainable weights of its own.

### 6.1 Module 15: Recommendation Scoring

A full-catalogue ranking per user, with every item the user already touched in training masked to $-\infty$ so it can never be re-recommended and mistaken for a discovery. The small heatmap on the diagram is an illustrative score matrix, not real output.

### 6.2 Module 16: Explanation Branch

This branch retrieves supporting evidence from six sources: shared category, shared brand, similar high-trust users, recent interaction history, semantic similarity and review sentiment. It fills a template only when at least one source returns something; otherwise it produces no explanation rather than a plausible one it cannot back up.

### 6.3 Module 17: Counterfactual Branch

For an item that just missed the top-K, this branch runs a bisection search over one perturbable factor at a time (category affinity, brand affinity, recency weight, trust weight or semantic similarity) and reports the size of change that would cross into the top-K, or an explicit INFEASIBLE result if nothing within bounds does. INFEASIBLE is a real answer, not a failure state to hide.

### 6.4 Module 18: Confidence Branch

Confidence blends three signals: predictive stability (weight 0.5), evidence volume from the user's interaction count (0.3) and evidence trust (0.2). Stability can come from Monte Carlo dropout or, as evaluated in the paper, from disagreement between independently trained ensemble members. The bar chart on the diagram shows that 0.50 / 0.30 / 0.20 split. The defining behaviour, enforced by a test, is that a cold-start user with one interaction must receive low confidence even when the raw score is high: confidence is deliberately decoupled from rank. Whether the score also tracks correctness is measured in the paper and reported as it came out.

## 7. Final Assembly

The wide green band is where the four branches converge into the one artefact the system exists to produce: a ranked list per user in which every item carries a score, a retrieved explanation, a counterfactual statement and a confidence value together.

### 7.1 The live example

The annotation under the band is a real output of the trained model, regenerated by `scripts/explain_example.py` rather than typed in: the user with the longest training history among evaluable users (117 interactions, behavioural trust 0.996) is recommended 'Sega Dreamcast Console'. 'Sega Dreamcast Console' was recommended because 9 high-trust users with overlapping history also bought it, it belongs to legacy systems, which dominates the user's history, and the user's recent activity is concentrated in legacy systems. Confidence 0.883 (117 training interaction(s), supporting-evidence trust 1.00). For the first item outside the top 5 ('Yakuza Kiwami 2: SteelBook Edition - PlayStation 4'), the counterfactual module reports: This item is ranked 6. It would enter the Top-5 if the user's semantic

similarity toward this product increased by 0.08 standard user-vector norms - the smallest change among the factors tested. The top item was not a hit in the held-out test split.

### 7.2 Module 39: Streamlit Dashboard

The final peach box, matching the Dataset box at the top, is the second endpoint. Its caption, "reads generated artefacts only: trains nothing, computes no metric of its own", is an architectural boundary. The dashboard is a read-only viewer over files the pipeline already wrote; if a number looks wrong in it, the fix is always upstream.

## 8. Tracing One Recommendation End to End

Following one recommendation through the diagram names the box responsible for each fact in the final explanation.

- **Dataset.** The user's interactions start as rows of Amazon Reviews 2023 (Section 1).
- **Ingestion, cleaning, split.** They survive schema adaptation and the k-core filter and land in the partition their timestamp dictates (Sections 2.1 to 2.3). Leakage Guards and EDA run alongside but contribute nothing forward.
- **Feature engineering.** Each review is read twice, for sentiment and for semantics, and fused into one 128-d vector (Section 3).
- **Trust-aware graph.** The same interactions become edges. The behavioural detector scores the user, giving the interaction-averaged trust that later feeds the explanation and confidence branches, and recency discounts older interactions (Section 4).
- **HGT encoder.** The graph and review vectors are encoded into a 128-d embedding for the user and each product, trained with a share of interaction edges hidden each epoch (Section 5).
- **Scoring.** A dot product produces the raw score that puts the item in the user's top-K (Section 6.1).
- **Explanation and counterfactual.** The explanation branch checks all six evidence sources and uses those that return something; the counterfactual branch reports what would change a near miss (Sections 6.2 and 6.3).
- **Confidence.** Stability, the user's evidence volume and the same trust figure combine into the confidence value (Section 6.4).
- **Final output.** Score, explanation, counterfactual and confidence are assembled into one record, and that same record is what the dashboard shows (Section 7).

Every box other than Leakage Guards and EDA appears in that chain, which is the point of drawing the architecture as one connected figure: a box either does work that reaches the number a user sees, or it is marked terminal so a reader never has to wonder which.